

Adaptive Traffic Signal Control Based on Multi-Agent Soft Actor-Critic

Hua Huang, *Member, IEEE*, Wei Zhang, *Student Member, IEEE*, Chuang Li, Xiaolong Li, Haidong Wang and You Wu, *Member, IEEE*

Abstract—In IoT-centric adaptive traffic signal control, existing multi-agent deep reinforcement learning methods are constrained by insufficient communication mechanisms. These limitations become particularly evident when handling the dynamic traffic networks and high-dimensional action spaces, leading to unstable environmental observability and diminished control effectiveness. Motivated by the idea of policy entropy maximization and the paradigm of centralized training and decentralized execution, the MASAC (Multi-Agent Soft Actor-Critic) algorithm was developed. However, the algorithm exhibited issues such as unstable training and low sample efficiency during its application. To enhance MASAC's training speed and stability, the prioritized experience replay was introduced, leading to the development of a Multi-Agent Soft Actor-Critic algorithm based on prioritized experience replay (MPER-SAC), which improves the model's learning efficiency. Joint simulation experiments using SUMO and Carla demonstrated that the proposed algorithm increased the average reward in the traffic environment by 9.55%, boosted average vehicle speed by 14.52%, and reduced average queue length and average intersection delay by 15.35% and 11.26%, respectively. Additionally, the ablation study displayed significant performance improvements, with an 11.36% increase in average reward, a 11.65% reduction in average queue length, and a 8.12% decrease in total travel delay. Additional performance metrics also exhibited varying degrees of enhancement.

Index Terms—Signal control, prioritized experience replay, multi-agent deep reinforcement learning, joint simulation

I. INTRODUCTION

IN recent years, the rapid economic growth and urbanization have significantly increased urban traffic. When traffic volumes [1] exceed road capacity during peak periods, it leads to congestion, which not only limits travel efficiency and economic development but also poses safety risks and contributes to environmental pollution [2], [3]. Consequently, research on intelligent urban transportation systems has consistently focused on reducing traffic congestion. Adaptive Traffic Signal

Control (ATSC) [4] is among the most used techniques for controlling traffic lights, supporting the development of smart city with IoT by ensuring smoother traffic flow at intersections and reducing urban congestion [5].

Autonomous timing modifications of the real-time signal are essential to reduce traffic congestion in ATSC. Traditional methods, such as Split Cycle Offset Optimizing Technique (SCOOT) and the Sydney Coordinated Adaptive Traffic System (SCATS) [6], rely on algorithms that are computationally intensive and inflexible, making them slow to adapt to the real-time traffic conditions. Although advancements like the Firefly Algorithm [7] and the combination of Decomposed Fuzzy Systems with Genetic Algorithms [8] have improved signal control, their effectiveness remains limited in large-scale traffic scenarios. This paper focused on intersection signal control and urban road simulation, with the support of the Xiangjiang Laboratory's urban smart city transportation research and development project. By tackling ATSC challenges through technological innovation, the project's goals are to lessen congestion and improve transportation efficiency [9].

Deep reinforcement learning (DRL) has become an essential tool in ATSC due to its iterative nature and ability to adapt to unexpected events, made possible by advances in reinforcement learning and deep learning technologies [10], [11]. DRL uses deep neural networks to approximate value functions. Approaches like Deep Q-Network (DQN) [12] and Deep Deterministic Policy Gradient (DDPG) [13] have shown significant progress in policy optimization. However, these methods are primarily designed for single-agent learning problems [14]. In complex traffic scenarios, collecting all relevant data can cause significant delays and increases the likelihood of data collection failures. Dimensionality's curse, where the number of potential states and actions grows exponentially, demands greater processing and communication capacity [15], which hampers global optimization [16], [17]. To address these challenges, researchers have adopted DRL to multi-agent systems, to promote the development of multi-agent deep reinforcement learning (MADRL). The Cooperative Double Q-Learning algorithm [18] provided a scalable independent double Q-Learning approach that used the double estimators and an upper confidence bound strategy. This approach facilitates discovery, curtailing the overvaluation bias typical of standard separate Q-learning. The Multi-Agent Advantage Actor-Critic (MA2C) [19] is a decentralized MADRL algorithm evolved from the Independent Advantage Actor-Critic (IA2C) method,

This work has been supported by Natural Science Foundation of Hunan Province (2023JJ50319), the Young Program of the Major Program of Xiangjiang Laboratory (23XJ01002), National Natural Science Foundation of China (72342018), Scientific Research Project of Hunan Provincial Department of Education (No. 24A0440), Major Project of Xiangjiang Laboratory (XJ2023001). (Corresponding author: Chuang Li)

Hua Huang, Wei Zhang, Xiaolong Li, Chuang Li, and Haidong Wang are with the School of Computer Science, Hunan University of Technology and Business, Changsha, 410205, People's Republic of China, and Xiangjiang Laboratory, Changsha 410000, People's Republic of China (e-mail: jdz_hh@qq.com; zhangweihutb@163.com; chuangli@hutb.edu.cn; xlli@guet.edu.cn; whd@hutb.edu.cn).

You Wu is with the School of WMG, University of Warwick, Gibbet Hill Road, Coventry, CV47AL, United Kingdom (e-mail: you.wu.8@warwick.ac.uk).

Copyright (c) 20xx IEEE. Personal use of this material is permitted. However, permission to use this material for any other purposes must be obtained from the IEEE by sending a request to pubs-permissions@ieee.org.

designed for the scalability. However, determining an appropriate discount factor that reduces the influence of state and reward information from other agents remains a challenge. In a multi-agent environment, agents’ actions are influenced by the policies of others, creating an unstable environment that violates the Markov property. Current MADRL algorithms do not fully address this issue. The Centralized Training with Decentralized Execution (CTDE) method [20] is gaining popularity in MADRL for effectively handling non-stationary environments in multi-agent systems [21].

The methods discussed above suffer from low sample efficiency and slow convergence rates. To address these issues and improve the efficiency of ATSC and decision-making in MADRL, this paper approach a novel function. It incorporates the average vehicle waiting time to guide agents in optimizing traffic flow and coordinating intersections. Building on the Soft Actor-Critic (SAC) algorithm [22] and specifically adapted for the MASAC algorithm trained using the CTDE approach [23], we put forward the MASAC with Prioritized Experience Replay (MPER-SAC) algorithm ,Its network architecture is shown in Fig.1. Lastly, Simulation experiments were conducted in a synthetic high-traffic density network, created using the Simulation of Urban Mobility (SUMO) software in combined with the Unreal Engine simulator Carla [24]. The results indicate our algorithm outperforms the existing MADRL methods in ATSC efficiency. The main contributions of this work are shown as below:

other state-of-the-art multi-agent deep reinforcement learning algorithms in terms of average queue length, average intersection delay, and average vehicle speed, as demonstrated by simulation results on large-scale synthetic traffic networks.

The remainder of the paper is organized as follows. Section II reviews the related Work. Section III formulates SAC Algorithm and MSAC Based on Traffic Signal Control. Section IV presents the implementation and optimization of traffic signal control based on MPER-SAC. Section V discusses the experimental results. Concluding remarks are described in Section VI.

II. RELATED WORK

Initial studies into standard traffic light management tactics heavily utilized expert-led techniques [25]. However, these approaches were limited by inaccurate traffic data collection and the constrained capabilities of traffic monitoring systems. With the development of DRL theory, it has gradually emerged as a leading research trend in ATSC. DRL [26] uses repeated experimentation to discover optimal decisions in unknown environments, effectively addressing sequential decision-making challenges [27], [28]. DRL-based ATSC methods are typically categorized as single-agent or multi-agent approaches, depending on the number of agents involved. Single-agent methods are generally used to manage traffic signals at individual intersections autonomously, while multi-agent methods are designed to coordinate the control of signals across multiple intersections.

A. Single-Agent Adaptive Traffic Signal Control

Liang et al. [29] recently conducted a simulation of traffic environments in SUMO, representing the conditions at a single intersection as a matrix and defining the four signal phases as the four actions output by the agent. The average waiting time difference between two successive time steps served as the basis for the reward function. Compared to conventional traffic signal control, which uses fixed phase sequences and durations, their Deep Q-network model optimized traffic light control, resulting in a 25.7% reduction in average intersection waiting time. Genders and Razav [30] used both the Asynchronous Advantage Actor-Critic (A3C) method and a distributed Q-learning method that combines neural networks to estimate function values. They used three different state representations to evaluate the performance of these algorithms, which varied in level of detail, from rough to refined. The findings indicated that using low-res data—like occupancy and average velocity—yielded comparable outcomes to high-res data—like individual car locations and velocities—when paired with a neural network that had a full connection. However, the best performance was achieved with deep neural networks and high-resolution state representations. Xu et al. [31] introduced a new controller based on Deep Q-Network (DQN) and proposed a targeted knowledge transfer framework. They used a batch learning framework in conjunction with a novel transfer learning model, using

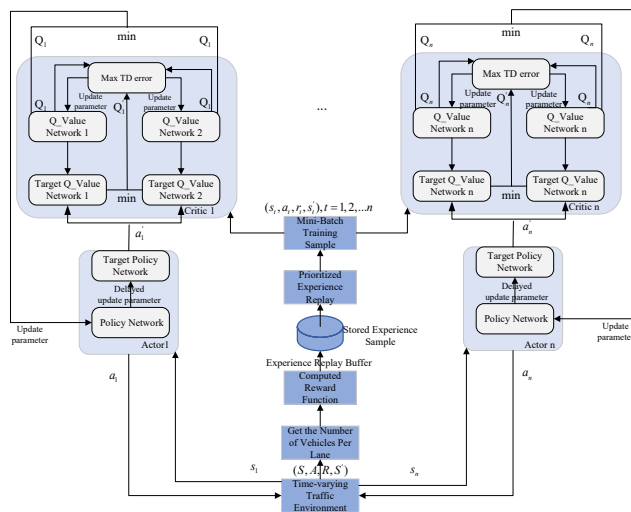


Fig. 1: MPER-SAC architecture diagram.

- 1) A new ATSC algorithm, MPER-SAC, is introduced for multi-intersection scenarios. This algorithm provides a stable training process, improving overall robustness and stability.
- 2) Prioritized Experience Replay is integrated into the MASAC algorithm in the DRL framework, allowing agents to explore state and policy spaces more comprehensively. This integration accelerates the agents' learning process and generates more valuable interaction data for training.
- 3) The use of the high-fidelity simulator Carla enhances simulation accuracy. The proposed method outperforms

both real data and synthesized uniformly distributed traffic flow data for experiments at independent intersections. Gao et al. [32] developed a signal control algorithm based on deep reinforcement learning and incorporated a target network and experience replay. The program extracts relevant attributes from unprocessed live traffic information and determines the most effective ATSC strategies.

B. Multi-Agent Adaptive Traffic Signal Control

The Modified Proximal Policy Optimisation (MPPO) algorithm, introduced by Yang et al. [33], enhances MADRL's ability to manage traffic signals and vehicles from a global perspective. Zhou et al. [34] leveraged the widespread availability of edge-enabled vehicular networks to develop an edge-distributed reinforcement learning traffic light control scheme, using a multi-agent Deep Q-network to implement adaptive traffic light control strategies. Ge et al. [35] proposed a method called Multi-Agent Transfer Soft Actor-Critic with Multi-View Encoder (MT-SAC) to improve traffic signal control performance. Zhu et al. [36] introduced A novel context-aware multi-agent broad reinforcement learning (CAMABRL) approach for adaptive traffic signal control is proposed. This method uses a broad learning system (BLS) in a flat network structure for decision-making, and leverages a long short-term memory (LSTM) network to capture the state information of adjacent agents and potential state information, enabling more far-sighted decision-making. Yang and Yang [37] introduced the concept of a semi-decentralized feudal multi-agent (SFM) framework to tackle the issue of burgeoning state spaces in modern multi-agent systems as the number of agents skyrockets. They further enhanced the framework by integrating the learned-goal soft actor-critic (LSAC) algorithm. This algorithm is adept at independently snatching and executing optimal hidden objectives within sub-policy layers, thereby boosting the efficacy of managing traffic signals at multiple intersections.

Independent Q-Learning (IQL) remains one of the most extensively studied methods in MADRL. Chu et al. [19] extended this approach to IA2C and introduced the multi-agent ATSC algorithm MA2C, which was based on this. This algorithm has proven effective in both synthetic and real-world traffic networks by incorporating a spatial discount factor to reduce the learning dimension and improving observability through fingerprint information from nearby intersections. However, the algorithm struggles to adapt to real-time traffic fluctuations under high traffic flow, limiting its ability to handle dynamic traffic changes effectively. Despite these advancements, the previously mentioned MARL methods did not adopt the CTDE framework, which can lead to unstable training, information asymmetry, and coordination difficulties. To address these challenges, A traffic signal control system based on the CTDE framework was suggested by Yan et al. [38]. With the help of this method, agents may dynamically modify their focus in response to traffic flow data, facilitating faster and more efficient adaptation to the traffic environment on a broader scale.

Single-agent deep reinforcement learning optimizes traffic signal control by observing the current traffic state, but it

overlooks the interdependencies between intersections [39]. Addressing congestion at a single intersection has limited impact in real-world applications. Current MADRL methods [40] experience slow convergence, extended training times and overestimated Q value. Addressing these complexities, the study proposes a groundbreaking MADRL method, MPER-SAC, drawing inspiration from prior techniques. The algorithm incorporates prioritized experience replay during training, using Temporal-Difference (TD) error to increase the likelihood of sampling and training on data with higher TD error. This approach promoted more efficient learning and accelerated the algorithm's convergence. Additionally, the challenge of sparse environmental rewards is mitigated by an improved reward function. The realism of the experiments is further enhanced by simulating urban road intersections using both SUMO(Fig.2b) and Carla (Fig.2c), validating the algorithm's effectiveness in controlling real-world dynamic traffic flows.

III. INTRODUCTION TO RELEVANT FUNDAMENTAL ALGORITHMS

This section begins with a summary of the fundamental concepts and formulas of the SAC deep reinforcement learning algorithm [41]. it then offered a detailed formalization of the MASAC algorithm within the CTDE framework. In MASAC, agents effectively mitigate the impact of traffic environment instability on performance by sharing observational and historical information.

A. SAC

The SAC algorithm [42], [43] marked a shift from traditional approaches that focus solely on maximizing agent returns. While conventional reinforcement learning focuses on maximizing cumulative expected reward, SAC goes further by aiming to increase the randomness of each action output. This dual objective promotes policy randomization, ensuring a more balanced distribution of probabilities across all possible actions, rather than favoring any specific one. The optimization objective is as follows:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha H(\pi(\cdot | s_t))], \quad (1)$$

where $\mathbb{E}$ represents the expected return for the current state, T denotes the cumulative time steps for each episode of agent-environment interaction, s_t and a_t are the state and action of the agent at time t , respectively. $r(s_t, a_t)$ is the reward value associated with the current state, $\rho_\pi(s_t, a_t)$ denotes the state-action distribution for the trajectory generated by the policy $\pi(a_t, s_t)$, $H(\pi(\cdot | s_t))$ at state s_t , is the entropy of the action under policy π , and α is a parameter that regulates the trade-off between giving priority to entropy and reward.

The SAC algorithm updates the actor network by reducing the Kullback-Leibler (KL) divergence, in accordance with the soft policy iteration technique. The objective to minimize is as follows:

$$J_\pi(\theta) = \mathbb{E}_{s_t \sim \mathcal{B}, a_t \sim \pi_\theta} [\alpha \log(\pi_\theta(a_t | s_t)) - Q_\beta(s_t, a_t)], \quad (2)$$

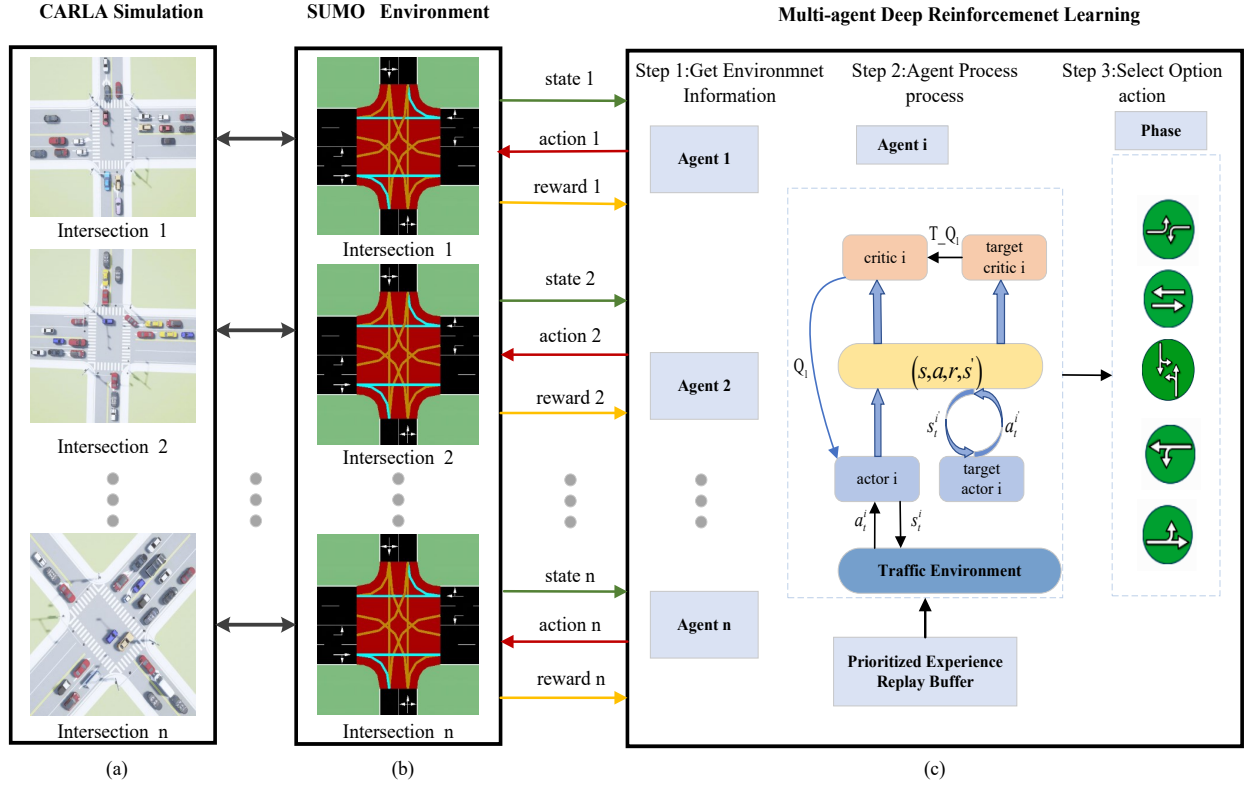


Fig. 2: Deep reinforcement learning framework diagram. (a) CARLA simulation scenario. (b) SUMO environment setup. (c) Multi-agent deep reinforcement learning network.

where θ and β denote the parameters of the agent's actor network π_θ and critic network Q_β , respectively. The estimated expectation of $J_\pi(\theta)$ is obtained through sampling from the experience replay buffer B, while $Q_\beta(s_t, a_t)$ in a_t can be obtained by sampling from the random policy π_θ . Therefore, the entire process cannot be updated through parameter backpropagation. To minimize $J_\pi(\theta)$ using stochastic gradient methods, the reparameterization trick can be utilized to rewrite the expectation. For instance, when sampling from a normal distribution, a stochastic noise variable ξ that follows a normal distribution can be added, transforming the original sampling process into:

$$\tilde{a}_\theta(s_t, \xi) = \tanh((\mu_\theta(s_t) + \sigma_\theta(s_t) \odot \xi)), \quad (3)$$

where $\tilde{a}_\theta(s_t, \xi)$ is a sample from the agent's action probability distribution $\pi_\theta(\cdot|s_t)$, μ_θ is the mean, σ_θ is the standard deviation. Using $\tanh$ ensures agent actions remain within defined limits. Reparameterization then proceeds thus:

$$\begin{aligned} \nabla_\theta J_\pi(\theta) &= \nabla_\theta \alpha \log(\pi_\theta(a_t|s_t)) + \\ &(\nabla_{a_t} \alpha \log(\pi_\theta(a_t|s_t)) - \nabla_{a_t} Q_\beta(s_t, a_t)) \nabla_\theta \tilde{a}_\theta(s_t, \xi), \end{aligned} \quad (4)$$

The objective of minimization that results is as follows:

$$J_\pi(\theta) = \mathbb{E}[\alpha \log(\pi_\theta(\tilde{a}_\theta(s_t, \xi)|s_t)) - Q_\beta(s_t, \tilde{a}_\theta(s_t, \xi))], \quad (5)$$

where, according to the maximum entropy principle, $a_t = \tilde{a}_\theta(s_t, \xi)$. The Bellman error minimization refines the critic network $J_Q(\beta)$:

$$J_Q(\beta) = \mathbb{E}_{s_t, a_t, r_t, s_{t+1} \sim B} \left[\frac{1}{2} (Q_\beta(s_t, a_t) - y)^2 \right], \quad (6)$$

$$\begin{aligned} y &= r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi_\theta(s_{t+1})} \\ &\left[Q_{\tilde{\beta}}(s_{t+1}, a_{t+1}) - \alpha \log(\pi_\theta(a_{t+1}|s_{t+1})) \right]. \end{aligned} \quad (7)$$

where y represent the target Q-value, γ represent the discount factor. (s_t, a_t, r_t, s_{t+1}) is a sample tuple obtained from the experience replay buffer B, the subsequent action $a_{t+1} \sim \pi_\theta(\cdot|s_{t+1})$ is sampled from the actor's distribution, and $\tilde{\beta}$ denotes the parameters of the target critic $Q_{\tilde{\beta}}$.

B. MASAC Based on Traffic Signal Control

The algorithm faces the challenges in converging in traffic scenario due to the instability inherent in multi-intersection and multi-vehicle environments, which amplify fluctuations in state transitions and reward values. While SAC can scale to complex, high-dimensional single-agent learning tasks [41], it is less suited for more intricate multi-agent learning tasks. To address this problem, the CTDE training framework has been adopted to extend SAC into MASAC [44]. in reinforcement learning, CTDE separates the training and execution phases. During training, agents improve the evaluation of the joint

policy by integrating actions, making comprehensive observations, and utilizing auxiliary information. In the execution phase, entities utilize regional data to govern their autonomous choices, applying the outcomes of centralized training to real-world traffic scenarios. The network model is shown in Fig.2(c). The actor network, the critic network, the target actor network, and the target critic network are the four deep neural networks that agent i is outfitted with. A thorough description of how the MASAC ATSC algorithm is implemented will be given in the sections that follow.

In the traffic scenario, there are n agents, with each agent being set as an intersection. The actor network of the agent is denoted as $\pi = \{\pi_1, \pi_2, \dots, \pi_n\}$, with parameters $\theta = \{\theta_1, \theta_2, \dots, \theta_n\}$. The π actor network's parameters are designated as θ , use π_{θ_i} to denote the parameters θ_i of the actor network π_i , while Q^{β_i} denotes the parameters β_i of the critic network Q_i . Training is based on the CTDE framework. To refresh the network for agent i , the specified objective must be reduced to its minimum:

$$J_{\pi_i}(\theta_i) = E_{s \sim B, \bar{a} \sim \pi_{\theta}}[\lambda \log(\pi_{\theta_i}(\bar{a}_i | s_t^i)) - Q^{\beta_i}(s, \bar{a})], \quad (8)$$

where λ is introduced as a hyperparameter to regulate the entropy weight, B as the experience replay buffer used to store the states, actions, rewards, and next states of all agents. A tuple (s, a, r, s') represent one stored entry in B. At any given time t , the state observations of all agents are denoted by $s = (s_t^1, s_t^2, \dots, s_t^n)$. The set of actions taken by each agent is represented by $a = (a_t^1, a_t^2, \dots, a_t^n)$, while the rewards for each agent are shown as $r = (r_{t,1}, r_{t,2}, \dots, r_{t,n})$. After the environment transitions to the next state, the new set of observations for all agents is denoted by $s' = (s_t^{1'}, s_t^{2'}, \dots, s_t^{n'})$. $Q^{\beta_i}(s, \bar{a})$ is an independent critic network for each agent i that takes the sum of all agents' states and actions as input. The actions $\bar{a}$ are taken from the experience replay buffer, and $\bar{a}_i$ is generated by the current policy of agent i to ensure stable training of the agent. Action $\bar{a}_i$ is sampled from the local policy $\bar{a}_i \sim \pi_{\theta_i}(\cdot | s_t^i)$; its expected return is encoded by the corresponding Q -value. Every agent autonomously selects and executes actions, guided by its private reward signal and its dedicated policy network conditioned solely on its own state. Each decentralized actor $\pi_{\theta_i}(\cdot | s_t^i)$ conditions its action distribution solely on its local observation $s_t^{i'}$. The agent then samples from this probability distribution to obtain the specific action $\bar{a}_i$. By using the reparameterization sampling technique, the objective can be represented in a differentiable form as follows:

$$J_{\pi_i}(\theta_i) = E[\lambda \log(\pi_{\theta_i}(\bar{a}_i | s_t^i)) - Q^{\beta_i}(s, \bar{a})], \quad (9)$$

where $s \sim B$. To ensure differentiability after sampling, we set $\bar{a}_i = \bar{a}_{\theta_i}(s_t^i, \xi)$, where $\xi \sim N(0, 1)$. The sampling operation is performed on the action probability distribution $\pi_{\theta_i}(\cdot | s_t^i)$. The expectation in $J_{\pi_i}(\theta_i)$ can be approximated by sampling from the experience replay buffer B. The stochastic gradient descent approach may then be used to update the actor network for agent i .

Secondly, the critic network of agent i is updated by minimizing the Bellman error specific to agent i :

$$J_Q(\beta_i) = E_{(s,a,r,s') \sim B}[\frac{1}{2}Q^{\beta_i}(s, a) - y_i]^2, \quad (10)$$

where y_i is the temporal difference target for agent i :

$$y_i = r_{t,i} + \gamma E_{a' \sim \pi_{\bar{\theta}_i}}[Q^{\bar{\beta}_i}(s', a') - \alpha \log(\pi_{\bar{\theta}_i}(a_t^{i'} | s_t^{i'}))], \quad (11)$$

where $\bar{\theta}_i$ and $\bar{\beta}_i$ freeze the target actor and critic parameters, while α governs the randomness of the optimal policy. To stabilize the learning performance of the agents, unlike SAC, target actor network $\pi_{\bar{\theta}_i}$ and target critic network $Q^{\bar{\beta}_i}$ are introduced. The input for $Q^{\bar{\beta}_i}$ encompasses the subsequent states s' that all agents encounter following the environmental shifts. Furthermore, a refers to the subsequent actions a' that each agent's targeted actor network conjures up for i , using the updated state s' as its guide.

The actor and critic networks, augmented by target actor and critic networks, contribute to the stabilization of learning. In contrast, during MASAC training, only the actor and critic networks are directly trained. The actor network and target actor network use the agent's current observation s_t^i and the next state's observation $s_t^{i'}$ respectively to generate the present action and the target action. The critic network takes as input the current observations s and actions a of all agents and outputs the Q -value Q_i corresponding to the action taken by agent i . The target critic network takes as input the next state observations s' and actions a' of all agents and outputs the Q -value T_Q corresponding to the target action of agent i . To ensure the algorithm runs smoothly and consistently, we conduct a gentle refresh of both the target actor and critic networks right after adjusting their parameters.

$$\bar{\theta}_i = \tau \theta_i + (1 - \tau) \bar{\theta}_i, \quad (12)$$

$$\bar{\beta}_i = \tau \beta_i + (1 - \tau) \bar{\beta}_i. \quad (13)$$

where τ is a hyperparameter controlling the proportion of the soft update, with $\tau \ll 1$; θ_i and β_i are the parameters of the actor network and critic network, respectively.

IV. IMPLEMENTATION AND OPTIMIZATION OF TRAFFIC SIGNAL CONTROL BASED ON MPER-SAC

The MPER-SAC algorithm's implementation in the urban traffic simulation tool SUMO is the subject of this section. The decision-making process for a single agent is typically described by a Markov Decision Process (MDP) [45] in deep reinforcement learning. In a multi-agent environment, the rewards and state transitions are influenced by the strategies and interactions of the agents, which are modelled using a Markov Game Process (MGP) [46]. Despite the fact that MGP expands upon MDP, its fundamental structure is the decision framework of MDP, which comprises the definitions of state, action, and reward functions. It is imperative to first standardise traffic data in order to guarantee the stability of MDP, which in turn improves the stability of neural network training. Consequently, the development of a suitable neural network architecture is an essential component of the implementation

of effective reinforcement learning. In conclusion, this paper introduces the Multi-Agent Prioritized Experience Replay Soft Actor-Critic (MPER-SAC), An innovative approach for MADRL. The agent training process achieves greater stability and faster convergence via prioritized experience replay [47], which prioritises the training of samples with larger estimation errors and poorer policy performance, thereby improving upon MASAC.

A. MDP Modeling for Traffic Signal Control

During the simulation time T_s , agent-environment interaction occurs in discrete intervals of Δt . Subsequently, an environment emulation is conducted for Δt seconds post each MDP iteration. If the time interval Δt stretches on for too long, the agent's flexibility might fall short. Conversely, when Δt is overly brief, the deep reinforcement learning's decision-making process might face delays due to the computational costs and communication hiccups. For this purpose, choosing Δt is crucial. Moreover, traffic accidents may result from vehicle collisions if the agent's control decisions are made too frequently. Therefore, for safety, a yellow duration $t_y < \Delta t$ is implemented at each signal transition. The objectives and methods of this study are similar to those in [48]. Thus, following the parameter selection by Aslani et al. [48], we set $\Delta t = 5s$ seconds and $t_y = 2s$ seconds, planning a total number of steps $T = T_s/\Delta t$. The agent selects one action in each reinforcement learning step for simulation experiments, with a duration of Δt .

1) *State Definition*: The definition of states is crucial for accurately describing the traffic conditions at an intersection. At any point in time t , every intersection agent i receives a local observation denoted as s_t^i . This s_t^i encompasses the current state of the traffic light and the local traffic conditions surrounding the intersection. All possible observations form the state space $s = (s_t^1, s_t^2, \dots, s_t^n)$:

$$s_t^i = \{AwT_t[l], wave_t[l]\} \quad j \in \epsilon, l \in L_{ji}. \quad (14)$$

where l signifies every input roadway for node i (the traffic controller), and L_{ji} encompasses both the in-flow and out-flow paths at the interchange. The total accumulated waiting time of vehicles during the current phase is measured by $AwT[s]$, while $wave[veh]$ counts any vehicles that are coming to the intersection within 50 meters of each lane. These metrics can be obtained using inductive loop detectors near the intersection, ensuring the real-time performance of ATSC. In SUMO, Lane Area Detectors are employed to gather the corresponding vehicle state information.

2) *Action Definition*: In traffic signal control research, various standard action definitions exist, such as phase switching, phase duration, and the phases themselves [49]. Our approach uses adaptive phases, where red and green signal fusion defines unique local functionalities. This approach facilitates more flexible and direct ATSC using deep reinforcement learning agents. Specifically, the agent is associated with the chosen signal phase at the traffic intersection, allowing the reinforcement learning agent to select and implement one of the actions during each step's duration. Unlike the fixed phase sequence

methods that either extend or terminate a phase, this method adjusts by extending or switching to other phases based on traffic flow fluctuations, potentially skipping unnecessary phases. Therefore, this definition primarily uses a non-cyclic timing scheme with variable phase sequences, where both the cycle length and phase sequence are not predetermined. Hence, at time step t , agent i controls the intersection traffic signal by selecting an action a_t^i , which determines the next effective phase p : $a_t^i = p, p \in \{1, 2, \dots, n\}$.

where n is the collection of all potential phases that the agent can access. It is noteworthy that if the selected action coincides with the current green light phase, the green light duration for this phase will be extended by a specified time interval (e.g., 1 second). Conversely, if the chosen course of action deviates from the present stage, the system will accumulate the elapsed time, subsequently switching the green light to phase a_t^i for the time interval t^k :

$$t^k = \begin{cases} G_{a_t^i}^{\min} + Y^{a_t^i} + R^{a_t^i}, & a_t^i \neq a_t^{i-1}, \\ 1s, & a_t^i = a_t^{i-1}. \end{cases} \quad (15)$$

where $G_{a_t^i}^{\min}$ is the minimum green light time for phase a_t^i , $Y^{a_t^i}$ is the yellow light time for phase a_t^i , and $R^{a_t^i}$ is the all-red time for phase a_t^i .

3) *Reward Definition*: The performance of ATSC is contingent upon the definition of rewards. Rewards that are appropriate for MADRL should be frequently measurable and spatially decomposable [50]. In this paper, the reward expression is defined as:

$$r_{t,i} = - \sum_{j \in \epsilon, l \in L_{ji}} (queue_{t+\Delta t}[l] + \alpha \cdot AwT_{t+\Delta t}[l]). \quad (16)$$

In the context of L , where i and j represent neighboring nodes, and α signifies the weight, the term $queue[veh]$ signifies the queue's length across each lane leading in. It's crucial to note that the primary objective of the DRL agent is to maximize cumulative rewards in the long run. $queue$ and AwT are measured at time $t + \Delta t$ since the reward is evaluated after the decision. This definition emphasizes both traffic congestion and travel delay, ensuring that the reward is directly correlated with the state and actions. The reward for each agent is encapsulated in the set $r = (r_{t,1}, r_{t,2}, \dots, r_{t,n})$.

B. DNN Architecture Design and Normalization Strategy

1) *Network Architecture*: Traffic flow is a multifaceted spatiotemporal dataset in actual traffic environments. When an agent simply accesses the present environmental condition, the MDP process could grow non-stationary. A common method for achieving MDP stability is to input historical states into the DNN. Yet this approach substantially elevates the state dimensionality, resulting in a dimensionality explosion and a decrease in the influence of recent traffic states on the DNN. Consequently, this paper introduces prioritized experience replay to resolve these concerns. A prioritized experience replay buffer $B = \{b_1, b_2, \dots, b_t\}$ is used to store the data from the agent-environment interactions, where $b_t = \{s_t^i, a_t^i, r_{t,i}, s_{t+1}^i\}$. The agent concentrates on learning from experiences that have a substantial impact on policy improvement during training

by sampling data from the experience buffer B. This method assists the agent in the formulation of more effective decision-making strategies.

2) *Normalization*: Normalization is a necessity for the training of DNNs. Therefore, to handle traffic environment data, this paper normalizes each *wave* and *AwT* state and clips them to the range [0, 2]. In order against gradient explosion, gradient clipping is employed, establishing a maximum value of 40 for each gradient. Additionally, reward are normalized and clipped to the range [-2, 2] to stabilize mini-batch updates. These measures significantly enhance the model's stability and training efficiency in complex traffic environments.

C. MPER-SAC Based on Prioritized Experience Replay

Under the MASAC framework, uniform sampling is employed to extract transition samples uniformly from the experience replay buffer. This classical reinforcement learning technique is used for sample sampling. Nevertheless, the effects of various experiences on training outcomes are not consistent, and even the same experience may yield different results in different training epochs. The current training may be significantly impacted by certain experiences, while others may not. Schaul et al. [51] introduced prioritized experience replay (PER), which has exhibited exceptional performance on deep Q-network. Our introduction of PER into MASAC was motivated by this. In a manner similar to Schaul et al. [51], we employ the TD-error as an importance metric to determine priorities. The subsequent function is employed by PER to determine priorities in DRL:

$$p(j) = \frac{\delta_j^\alpha}{\sum_{i=1}^k \delta_j^\alpha}, \quad (17)$$

$$\delta_j = |Loss_j| + \varepsilon, \quad (18)$$

where α is the control coefficient in the prioritization, and δ_j^α is the weight coefficient of sample j , δ_j represents the priority of experience sample j , and ε represents a minimal, positive quantity to circumvent zero-probability occurrences. Given the combined loss in critic and actor updates is often small, it is advantageous to use the sum of the agents' TD-error as the priority. The priority δ_j , which is the total of each agent's absolute TD-error, can be represented as:

$$\delta_j = \sum_{i=1}^n |Loss_j(i)| + \varepsilon, \quad (19)$$

In MPER-SAC, the TD-error needs to additionally consider the entropy term in its calculation. Therefore, the TD-error for agent i is calculated as follows:

$$Loss(i) = \mathbb{E}_{(s,a,r,s') \sim B} [(r_{t,i} + \gamma \mathbb{E}_{a' \sim \pi_{\bar{\theta}_i}} (Q^{\bar{\beta}_i}(s', a')) - \alpha \log(\pi_{\bar{\theta}_i}(a'_t | o'_i))) - J_Q(\beta_i)]. \quad (20)$$

where $Loss_j(i)$ represents the TD-error of all agents, and the TD-error for agent i is computed through this $Loss(i)$.

Prioritized experience replay assigns a priority to each sample, thereby increasing the likelihood that samples with

Algorithm 1: Algorithm of MPER-SAC

Input: Maximum number of iterations M , mini-batch size $|B|$, episode size η , step size ω , replay period K , learning rate α

Output: $\bar{\theta}_i, \bar{\beta}_i$

- 1: Initialize Actor network is $\pi = \{\pi_1, \pi_2, \dots, \pi_n\}$, the Critic network $Q = \{Q^{\beta_1}, Q^{\beta_2}, \dots, Q^{\beta_n}\}$, the target Actor network is $\pi_{\bar{\theta}_i} \leftarrow \pi_{\theta_i}$, and the target Critic network is $Q^{\bar{\beta}_i} \leftarrow Q^{\beta_i}$
- 2: Initialize the experience buffer B and the number of learning cycles, priority $\delta_1 = 1$
- 3: **for** $i = 1, \dots, \eta$ **do**
- 4: Reset the environment and obtain the initial state value s_t^i of agent i
- 5: **for** $j = 1, \dots, \omega$ **do**
- 6: Select action $a_t^i \sim \pi_{\theta}(\cdot | s_t^i)$ for each agent i
- 7: Obtain the corresponding reward $r_{t,i}$ and the next state s_{t+1}^i
- 8: Store the experience sample (s, a, r, s') into the experience pool B with the highest priority $\max_{i < K} \delta_i$ for sampling and training
- 9: **for** Each learning cycle is as follows: **do**
- 10: According to the PER mechanism, sample experiences from the experience replay pool
- 11: Uniformly sample from $[0, \delta_{\max}]$ and update the Critic network
- 12: Calculate the TD-error values for the sampled experiences and update their priorities δ_k using the formula $\delta_k = \sum_{i=1}^n |Loss_k(i)| + \varepsilon$
- 13: Update the Actor network
- 14: **end for**
- 15: Update the target network parameters
- 16: $\bar{\theta}_i = \tau \theta_i + (1 - \tau) \bar{\theta}_i$
- 17: $\bar{\beta}_i = \tau \beta_i + (1 - \tau) \bar{\beta}_i$
- 18: Periodically update the experience data in the global experience replay pool
- 19: **end for**
- 20: **end for**

higher priorities are selected from the experience pool. In order to simplify the sorting process, the priority of state transitions for all samples is stored in a SumTree data structure. Proportional priority sampling is implemented in a mini-batch of size k . This method divides the interval $[0, \delta_{\max}]$ into k segments and implements uniform sampling within each segment. This method prevents overfitting and guarantees priority sampling rates. The loss values for other experiences in the replay buffer are not recalculated, as the algorithm's computational complexity ensures that the loss for the selected mini-batch experiences is only updated during training. The highest priority is given to new state transitions to guarantee that they are sampled preferentially.

V. EXPERIMENTS

The ATSC algorithm based on MPER-SAC will be evaluated in a simulated traffic environment through a collaborative simulation between SUMO [52] and Carla¹. A urban traffic simulation grid will be created for this purpose. The traffic simulation map will be exported using OpenStreetMap (OSM)², a collaborative project aimed at creating an editable map of the world, following established research methodologies. This allows the export of traffic simulation maps as actual road network files. The OSM file will then be converted to XODR (OpenDRIVE)³ format to be loaded into Carla, an open-source simulation platform built on Unreal Engine. The XODR file can be easily modified with RoadRunner⁴ to accommodate the necessary changes in the real road network. Finally, a script will be developed to enable synchronised simulation interaction between Carla and SUMO by utilising the Python API [53]. This section creates a highly realistic joint simulation environment based on Carla and SUMO to compare and analyze the various MADRLs' performance. As shown in Fig.3.

A. Scene Setting

A traffic scenario that has been simulated is illustrated in Fig.3. The two main roadways are a single-lane road with an 11 m/s speed restriction and a two-lane road with a 20 m/s speed limit. Each intersection comprises five feasible lane phase alternatives, as illustrated in Fig.2c under the heading "Phase." Including straight-through movements and left turns for east-west directions, as well as straight-through and left turns for all four directions (east, west, north, and south). Right turns are set as the default movement. Four time-varying traffic flows were simulated in the simulation scenario, as illustrated in Fig.4, to increase the complexity of the MDP problem. The simulation was conducted for a duration of 3600 seconds. Flow 2 and Flow 4 are relatively large, while Flow 1 and Flow 3 are smaller. Origin-Destination (O-D) pairs are employed to generate the primary traffic flows, as illustrated in Fig.3c. In particular, Flow 1 includes all of the following directions: x7-x1, x8-x2, and x9-x3. Flow 2 is comprised of the following directions: x4-x10, x5-x11, and x6-x12. Flow 3 is comprised of the reverse directions x1-x7, x2-x8, and x3-x9, and Flow 4 is comprised of the following directions: x10-x4, x11-x5, and x12-x6. It has been noted that Flow 1 and Flow 2 commence with lower volumes at 1200 seconds, while the reverse directions, Flow 3 and Flow 4, starting the process of generating traffic flow. It essential to note that these flows are exclusively employed to simulate traffic demand, and the routes that individual vehicles take are generated at random.

B. Experimental Setup

Experiments involving advanced benchmark methods were executed to showcase MPER-SAC's reliability and perfor-

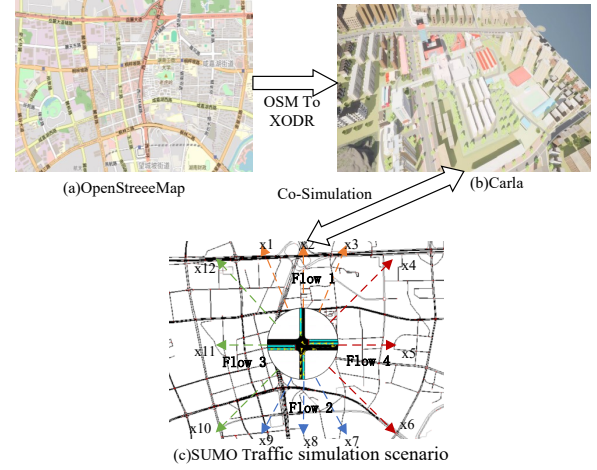


Fig. 3: Traffic simulation scenario.

mance efficacy. IQL-LR is a decentralised version of Independent Q-Learning [54], utilizing linear regression (LR). In contrast, IQL-DNN extends IQL by employing neural networks [54]. The primary difference between IQL-DNN and IQL-LR lies in how the Q-function is fitted: IQL-DNN uses deep neural networks (DNN) for Q-function fitting, while IQL-LR employs linear regression. The IA2C [19] algorithm extends the A2C framework, incorporating IQL principles. The key distinction between IQL and IA2C is that agents use different algorithms for training. The MA2C algorithm [19], an enhanced version of IA2C, shows significant performance improvements compared to IA2C. Additionally, MA2C employs two control strategies during training to improve the evaluation of ATSC performance compared to the base IA2C method.

The algorithms that were evaluated in this study were trained for nearly one million steps, which included 1400 training episodes, each of which consisted of $T = 720$ steps. Subsequently, the algorithms were subjected to multiple rounds of algorithm performance testing. Random seeds are essential for the generation of a set of random numbers that are used throughout the entire training and evaluation process. The training and evaluation sets are generated by differing random seeds; however, the same set contains identical seeds. The discount factor $\gamma = 0.99$ and the prioritisation exponent $\alpha = 0.75$ were utilised in our experiments. The learning rate $\eta_\theta = 10^{-4}$, the mini-batch size $|B| = 120$, and the replay buffer size are 1000 for MPER-SAC. Some parameters are summarized in Table I.

The algorithm's performance will be comprehensively evaluated in this article using five evaluation metrics, which are introduced as follows:

Average queue length: The control performance of traffic signals is typically evaluated by the average number of halted vehicles (veh) in each direction of the simulation scenario. Speed less than 0.1 m/s classify a vehicle as stationary.

$$avg_queue = \frac{1}{l} \sum_{i=1}^l queue_{t+\Delta t}(l). \quad (21)$$

Average intersection delay: The mean delay for all vehicles across the simulated traffic network's intersection zone,

¹<https://carla.org>

²<https://www.openstreetmap.org>

³<https://odrviewer.io>

⁴<https://www.mathworks.cn/products/roadrunner>

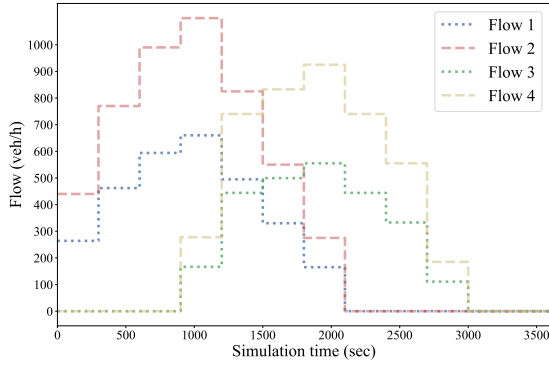


Fig. 4: Time-varying traffic flow.

TABLE I: Simulation Parameter Setting.

Parameters	Value
Steps	1e-6
Episodes	1400
T	720
Seeds	1,2...
γ	0.99
α	0.75
η_θ	10-4
$ B $	120
replay buffer size	1000

encompassing a 50m radius in this study.

$$avg_wait = \frac{1}{totcar} \sum_{i=1}^{totcar} wait_time(i). \quad (22)$$

where $totcar$ is the aggregate vehicle count, and $wait_time(i)$ represents vehicle i 's simulated delay.

Average vehicle speed: All vehicles in the complete traffic network of the simulation scene. The speed is obtained by the simulation function.

$$avg_speed = \frac{1}{totcar} \sum_{i=1}^{totcar} speed(i). \quad (23)$$

where $speed(i)$ denotes the speed of the i -th vehicle.

Trip completion flow: Mean frequency of vehicles finishing journeys within the traffic network's simulation phase, that is, the average number of vehicles num_outcar that arrive at the destination per second, is obtained through the simulation function.

$$trip_over = \frac{1}{T} \sum_{t=0}^{T-1} num_outcar. \quad (24)$$

Average travel delay: Mean waiting duration for all vehicles across the simulated traffic network. This metric is obtained from the tripinfo file, which is part of the simulation results generated by SUMO. The variable $waitingTime$ represents the travel delay time.

$$waitingTime = \sum_{i=1}^l AwT_{t+\Delta t}[l]. \quad (25)$$

The average intersection delay is different from the average travel delay. The average travel delay is a measure of the average waiting time of vehicles throughout their entire journey, the average intersection delay reflects the typical vehicle wait time at traffic intersections.

Furthermore, the average reward of the evaluation set is defined as: in order to more accurately characterize the performance of the entire algorithm.

$$\bar{R} = \frac{1}{T} \sum_{t=0}^{T-1} \left(\sum_{i \in n} r_{t,i} \right). \quad (26)$$

with n denoting the count of agents and T representing the loop's iterations.

C. Comparative Experiment

The performance of ATSC in the simulation scenario was evaluated using several key metrics, as shown in Table II. These metrics were collected during each phase of the experiment. The results indicate that the average reward for IQL-DNN, IQL-LR, IA2C, MA2C, and MPER-SAC were -2951.30, -1663.34, -433.03, -419.67, and -379.59, respectively. It is clear that MPER-SAC outperforms the other methods in terms of the specified objectives. MPER-SAC demonstrated a 9.55% improvement compared to the most effective baseline algorithm, MA2C. Additionally, the average queue length and average intersection delay decreased by 15.35% and 11.26%, respectively, while the average vehicle speed increased by 14.52%. Furthermore, the total trip completion flow across the entire simulation was within 1.59% of the best-performing MA2C method, and the overall average travel delay was 37 seconds longer (15.04% increase) compared to the best-performing IQL-LR method.

The training curves for each algorithm are shown in Fig.5, where the solid lines represent the trend of average reward for each training set, and the shaded areas reflect the changes in their standard deviations. As depicted in the graph, MADRL gradually learns from its accumulated experiences and ultimately reaches a local optimum. Initially, the training curves rise and then tend to converge over time. In Fig.5, IQL-DNN struggles to converge due to overfitting of the Q-function caused by partially observed states, which also accounts for its poor average reward performance. Although IQL-LR eventually converges, its results are relatively poor compared to other algorithms, mainly due to its inability to cope with environmental instability from fluctuating traffic flow. In contrast, MA2C demonstrates more stable learning capabilities than IA2C, with both achieving satisfactory convergence. Compared to IA2C and MA2C, MPER-SAC shows the most robust and effective overall learning performance. As more training iterations are used, the variance in average reward values gradually decreases, as seen in the shaded areas of the figure. This highlights the algorithm's strong convergence and stability in the later stages. Fig.6 shows that the evaluated episode rewards (rewards obtained at each simulation time step) align closely with the training results.

To further highlight the performance advantages of MADRL in ATSC, a comprehensive comparison of the proposed

TABLE II: Performance of ATSC in simulated environments.

Evaluation Metrics	MPER-SAC	MA2C	IA2C	IQL-LR	IQL-DNN
Average reward	-379.59	-419.67	-433.03	-1663.34	-2951.30
Average queue length[veh]	1.82	2.15	2.22	4.00	4.30
Average intersection delay[s/veh]	13.95	15.72	22.78	96.59	274.94
Average vehicle speed[m/s]	2.76	2.41	2.18	2.11	1.21
Trip completion flow[veh/5min]	0.62	0.63	0.53	0.57	0.20
Average travel delay[s]	283	309	359	246	332

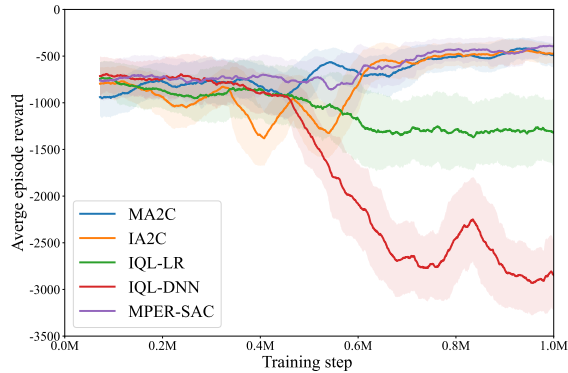


Fig. 5: Training curve for algorithms in traffic simulation scenario.

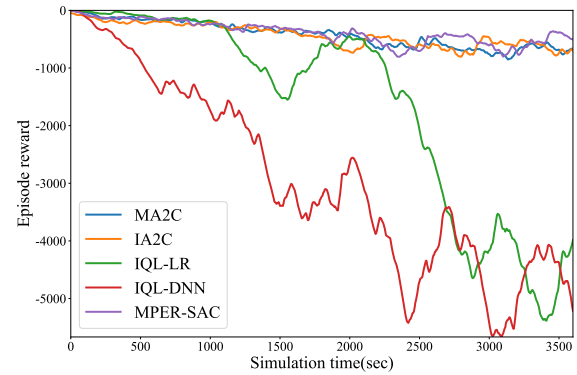


Fig. 6: Episode rewards in traffic simulation scenario.

MPER-SAC algorithm with other algorithms was conducted across five key metrics: average queue length, intersection delay, average vehicle speed, trip completion flow, and average travel delay. The trend of average queue length at intersections over each simulation step is clearly depicted in Fig.7. Both IQL-LR and IQL-DNN fail to develop stable control strategies to effectively manage traffic congestion. However, IA2C and MA2C perform well in reducing queue lengths, with MA2C outperforming IA2C, consistent with previous research findings. In contrast, MPER-SAC implements strategies that are typically more stable and sustainable, ensuring that queue lengths remain within a reasonable range at all times. The average queue length consistently stays below 3 vehicles, demonstrating exceptional stability and reliability.

The variation in average intersection delay for the entire traffic network during the simulation period is illustrated in Fig.8. As expected, IQL-LR and IQL-DNN show poor performance in managing intersection congestion, as they struggle to efficiently address obstructions, leading to increased delays. Additionally, as the simulation progresses, delay times in certain sections tend to increase. In contrast, while delays rise slightly due to increased traffic volume, IA2C and MA2C maintain relatively stable performance. However, MPER-SAC significantly reduces intersection delays, even during periods of high traffic volume. The proposed approach decreases the average intersection delay by 11.26% compared to the best-performing MA2C.

The trend of average vehicle speed across the entire traffic network over the simulation period is shown in Fig.9. As anticipated, IQL-LR and IQL-DNN fail to recover from congestion due to ineffective convergence during training, resulting in

relatively low average vehicle speeds. While IA2C achieves significantly higher speeds than MA2C at certain points during the simulation, MA2C excels in evenly distributing traffic flow across intersections by coordinating and sharing neighborhood information, ensuring smoother traffic flow. MPER-SAC efficiently controls the dynamic fluctuations of the complex traffic environment, particularly during peak traffic flow, by leveraging its maximum entropy strategy. The proposed algorithm increases average vehicle speed by 14.52%, maintaining an overall speed between 2-4 meters per second, thus meeting the high-speed requirements for efficient urban traffic.

Fig.10 and Fig.11 show the average trip delay and trip completion flow, respectively, over the simulation period for various ATSC strategies. Some shortcomings in MPER-SAC's performance, as revealed by the comparative analysis, are likely due to the road network layout and uneven lane distribution, factors that are inherent to the scenario settings. The trip completion flow is only 0.01 vehicles per second lower than the best-performing strategy, MA2C, indicating strong overall performance. Moreover, MPER-SAC shows a substantial improvement in trip completion flow compared to other benchmark algorithms, particularly during periods of high traffic volume. However, the average trip delay is 37 seconds longer than that of the best-performing IQL-LR, suggesting relatively weaker performance in this metric. Notably, IQL-LR's average trip delay exhibits significant fluctuations in the latter half of the simulation, despite its strong initial performance, likely due to the instability of linear programming. In contrast, MPER-SAC demonstrates exceptional stability, particularly during high traffic volume, showing the most consistent performance and the lowest average trip delay.

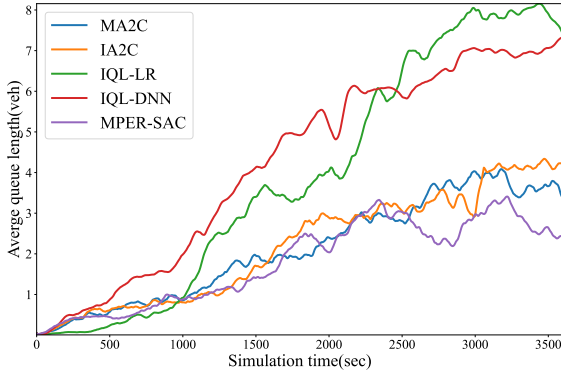


Fig. 7: Average queue length in simulation scenario.

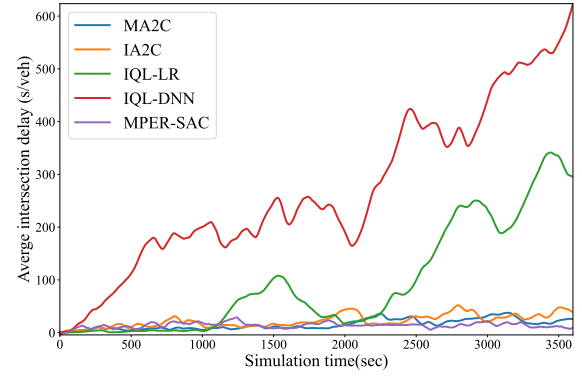


Fig. 8: Average intersection delay in simulation scenario.

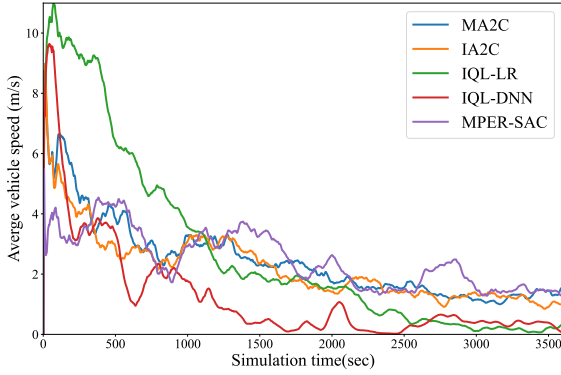


Fig. 9: Average vehicle speed in simulation scenario.

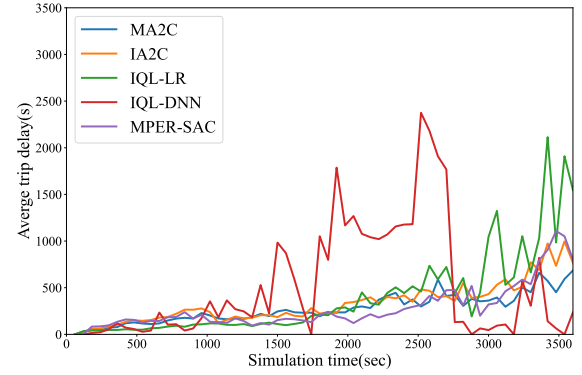


Fig. 10: Average trip delay in simulation scenario.

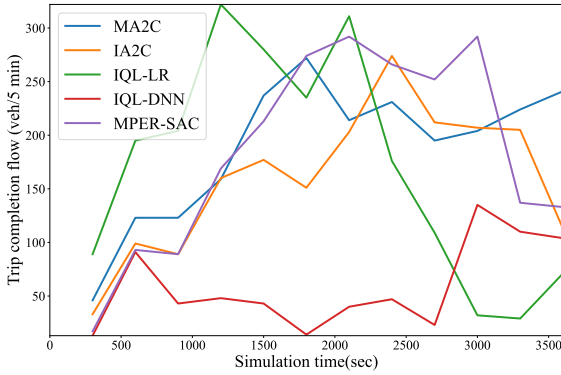


Fig. 11: Trip completion flow in simulation scenario.

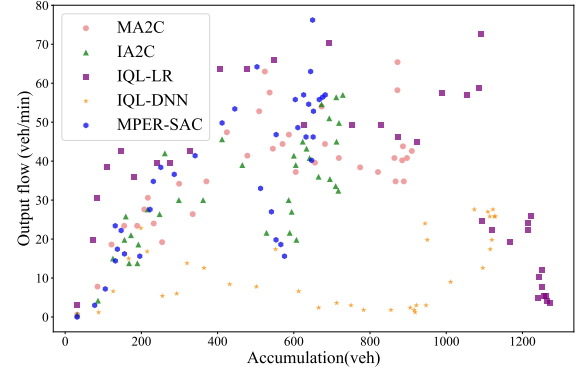


Fig. 12: Output traffic flow and cumulative vehicle dispersion in simulation scenario.

Fig.12 illustrates the relationship between the cumulative number of vehicles in the traffic network and trip completion flow for various ATSC strategy algorithms. The macroscopic fundamental diagram (MFD) for each algorithm shows that output increases with vehicle accumulation when traffic network density is low. However, as the network becomes saturated, output decreases, leading to congestion as more vehicles accumulate. While IQL-LR shows higher output flows in certain regions, these results are relatively scattered and do not represent an optimal solution. In contrast, MPER-SAC keeps most points near the optimal range, maximizing the utilization of the traffic network's capacity and ensuring smoother trip completions.

D. Ablation Experiment

The performance of the MASAC algorithm without prioritized experience replay (PER) and the MPER-SAC algorithm with PER was examined in this research, conducting ablation experiments in the same simulation environment. It is important to note that the tests and environment settings for both algorithms were kept consistent. As shown in Table III, PER demonstrated superiority in terms of the average reward, average queue length, and average intersection delay. However, it was relatively less effective in improving the trip completion flow and reducing the trip delays. Previous research [55] has indicated that the PER method can enhance reinforcement learning by improving policy exploration, which aids in developing a more optimal policy. Integrating PER into

TABLE III: Performance comparison with and without priority experience replay.

Evaluation Metrics	MPER-SAC	MASAC
Average reward	-379.59	-428.26
Average queue length[veh]	1.82	2.06
Average intersection delay[s/veh]	13.95	15.13
Average vehicle speed[m/s]	2.76	2.44
Trip completion flow[veh/5min]	0.62	0.63
Average travel delay[s]	283	308

MADRL helps prioritize the selection of experiences, enabling more efficient learning of optimal control strategies. According to this investigation, the MPER-SAC algorithm increased the average reward by approximately 11.36%, reduced the average queue length by 11.65%, and shortened overall trip delays by nearly 8.12%. Other performance metrics also showed varying degrees of improvement, the results showed an 8.00% improvement in average intersection delay and a 13.11% increase in average vehicle speed, while the trip completion flow saw a more modest rise of 1.59%.

Fig.13a compares the average queue length across the entire simulation scenario between the introduction of PER (MPER-SAC) and its absence (MASAC). The introduction of PER led to improved performance, with the average queue length consistently maintained at or below 3.25 vehicles. Moreover, it effectively keeps the queue length low even under high traffic volumes. Fig.13b illustrates the comparison of average intersection delay with and without PER. MPER-SAC significantly reduces intersection delays, showing a smooth overall

trend. In contrast, MASAC struggles to make effective policy decisions under high traffic volumes, the delay even exceeded 35 seconds per vehicle, resulting in increased congestion and delays at intersections.

In the continuous simulation scenario, Fig.13c and Fig.13d present the performance comparisons of average vehicle speed and episode rewards, respectively, with and without the introduction of PER. The curves clearly demonstrate that MPER-SAC, with the PER mechanism, shows significant improvement over MASAC in managing average vehicle speed and simulation step rewards. Although fluctuations may occur due to increased traffic volume, the overall control effectiveness of MPER-SAC remains superior to that of MASAC.

Fig.13e and Fig.13f compare the performance of trip completion flow and average trip delay with and without the introduction of PER. The results indicate that the implementation of PER efficiently manages high traffic volumes, ensuring a steady increase in trip completion flow. During the first 3000 seconds of the simulation, MPER-SAC shows significantly lower average latency than MASAC, with fewer fluctuations. Although its performance may fluctuate in response to environmental changes, MPER-SAC generally meets fundamental performance standards, demonstrating a high level of adaptability.

E. Algorithm Performance Analysis

Relying on the prior investigation, it was revealed that IQL-DN and IA2C performed badly in this case. Fig.14 further depicts the variations in gradient norm of the algorithms during the training phase. With relatively minor variations

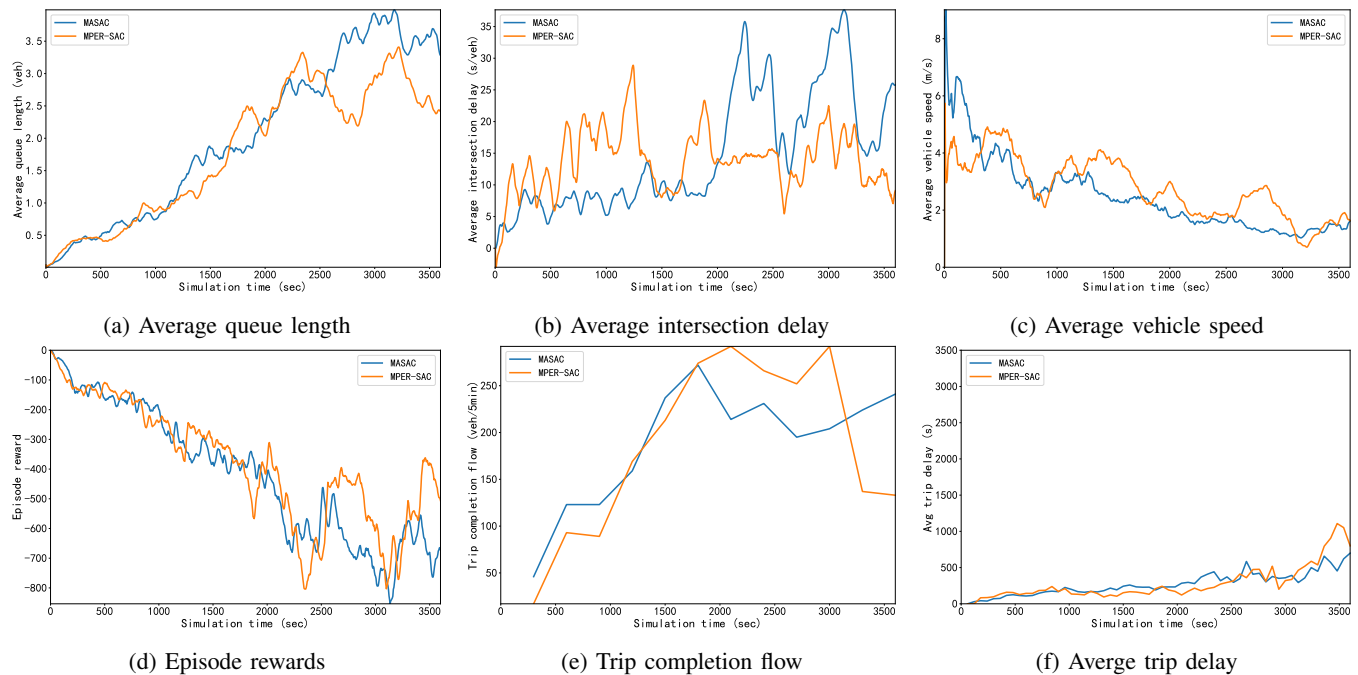


Fig. 13: Ablation experiments with priority experiential replay. (a)Average queue length in ablation experiment. (b)Average intersection delay in ablation experiment. (c)Average vehicle speed in ablation experiment. (d)Episode rewards in ablation experiment. (e)Trip completion flow in ablation experiment. (f)Average trip delay in ablation experiment.

during times of heavy traffic, the gradient norm of MPER-SAC demonstrates a steady convergence pattern and relative stability throughout the process. The magnitude of these variations is less than that of the baseline approaches. This pattern suggests that MPER-SAC uses PER to successfully address the issue of gradient explosion or disappearance. The algorithm may give priority to replaying experience relating to signal modifications at busy crossings during peak hours, which allows the policy to rapidly converge to the best answer. In contrast, other algorithms, although showing some stability, exhibit larger fluctuations overall, indicating weaker gradient control capabilities. The training step axis highlights the efficiency of MPER-SAC in achieving stable parameter updates, which is crucial for multi-agent collaborative tasks.

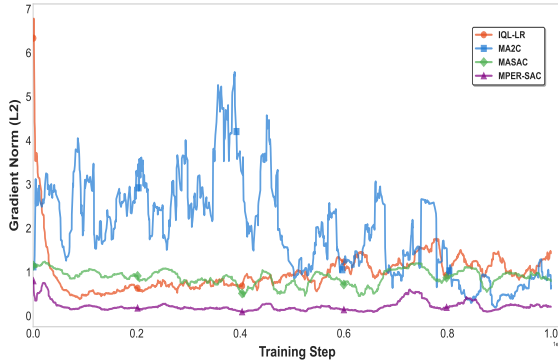


Fig. 14: Gradient norm.

The training times of many multi-agent deep reinforcement learning models are shown in Fig.15 to compare the training performance of various techniques. In comparison to the top-performing baselines, IQL-LR (36.03 hours) and MASAC (35.78 hours), the suggested MPER-SAC completes the identical training job in only 31.29 hours, which has the decrease of 13.16% and 12.55%, respectively. Three major breakthroughs are mostly responsible for this efficiency gain: First, high-value samples are intelligently filtered by the PER technique, which successfully reduces unnecessary calculations by around 30%. Second, the exceptional stability of gradient updates (with fluctuation range controlled within ± 0.05) prevents training interruptions caused by gradient anomalies. Third, the multi-agent coordination optimization strategy reduces cross-node communication overhead by 15%. These results demonstrate that MPER-SAC achieves superior training efficiency while maintaining competitive performance, well-suited for extensive multi-agent systems under strict computational limitations. (All experimental data were collected under identical hardware configurations with five repeated runs to ensure statistical significance.)

F. Experimental Results

Based on the analysis of the simulation results, it can be concluded that the ATSC system based on the MPER-SAC algorithm significantly alleviates intersection congestion. Compared to the best-performing benchmark algorithms, the overall average reward increased by 40.08, and the average vehicle speed improved by 0.35 meters per second. Additionally,

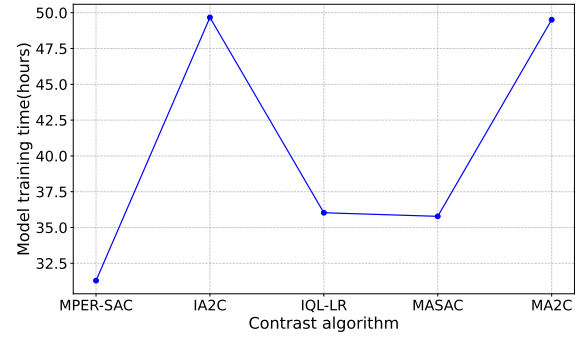


Fig. 15: Training performance.

the average queue length at intersections decreased by 0.33 vehicles, and the average intersection delay was reduced by 1.77 seconds per vehicle, however, the overall average trip delay differs by 37 seconds, and the trip completion flow differs by 0.01 vehicles. By incorporating prioritized experience replay, the training speed and stability of the algorithm were further enhanced: the average reward increased by 48.67, the average vehicle speed rose by 0.33 meters per second, the average queue length decreased by 0.24 vehicles, and the average intersection delay was reduced by 1.18 seconds per vehicle. Although the overall trip completion flow decreased by 0.01 vehicles per second and the average trip delay was shortened by 25 seconds, these results further validate the effectiveness of the MPER-SAC algorithm for ATSC.

VI. CONCLUSIONS AND FUTURE WORK

This paper proposes MPER-SAC, a novel MADRL approach that integrates the prioritized experience replay for adaptive traffic signal control. The algorithm extends the soft actor-critic framework to multi-agent settings using the centralized training with decentralized execution paradigm, significantly enhancing the training efficiency and stability. Extensive evaluations on a high-fidelity simulation platform integrating SUMO and Carla demonstrate the superior performance of our method in multi-intersection scenarios with time-varying traffic flows. Ablation studies confirm the critical role of prioritized experience replay in improving the learning efficacy. The proposed solution has been successfully deployed in Xiangjiang Laboratory's smart city initiative, effectively alleviating intersection congestion in simulated urban environments.

Future work will focus on testing the algorithm in more real-world traffic scenarios to further optimize its training performance, ensuring it can achieve the same traffic signal control effectiveness in more complex environments. The realism of traffic simulators will be progressively enhanced, and experiments will incorporate real-time dynamic traffic data as well as V2X data. Additionally, efforts will be made to improve the algorithm's robustness against noise and delayed state measurements from road sensors. Where necessary, pilot testing shall be carried out within the realistic traffic system scheduled for future implementation in the laboratory. As computational power advances, simulations will expand from

individual areas to city-wide complex scenarios. The ATSC method based on deep reinforcement learning will continue to evolve and ultimately be implemented to solve real-world traffic challenges.

REFERENCES

- [1] Y. Fei, K. Long, L. Xing, X. Pei, X. Li, and L. Yao, "Safety performance analysis of toll plaza diverging area based on an improved simulation platform for weak-constraint driving behaviors," *Accident Analysis & Prevention*, vol. 220, p. 108177, 2025.
- [2] M. Fadda, M. Anedda, R. Girau, G. Pau, and D. D. Giusto, "A social internet of things smart city solution for traffic and pollution monitoring in cagliari," *IEEE Internet of Things Journal*, vol. 10, no. 3, pp. 2373–2390, 2023.
- [3] C. Li, Y. Peng, G. Liu, Y. Li, X. Yang, and C. Chen, "Efficient vision transformer for human-centric aiot applications through token tracking assignment," *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 1029–1039, 2024.
- [4] S. Cai, J. Fang, and Mengyun, "FairSignal: A multi-agent reinforcement learning approach considering fairness for multi-intersection traffic signal control," *IEEE Internet of Things Journal*, pp. 1–1, 2025.
- [5] X. Chen, J. Zhang, and Y. Wang, "Urbanization and carbon efficiency: Causal evidence from china's city-county merger policy," *crossref*, 2024.
- [6] J. Luk, "Two traffic-responsive area traffic control methods: Scat and scout," *Traffic engineering & control*, vol. 25, no. 1, 1984.
- [7] C. Liu, Y. Ren, and X. Bi, "Timing optimization of regional traffic signals based on improved firefly algorithm," *Control and Decision*, vol. 35, no. 12, pp. 2829–2834, 2020.
- [8] R. Li and S. Xu, "Traffic signal control using genetic decomposed fuzzy systems," *International Journal of Fuzzy Systems*, vol. 22, no. 6, pp. 1939–1947, 2020.
- [9] C. Chen, K. Li, Y. Li, and X. Zou, "Regnn: A redundancy-eliminated graph neural networks accelerator," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 429–443, IEEE, 2022.
- [10] R. Zhu, S. Wu, L. Li, W. Ding, P. Lv, and L. Sui, "Adaptive broad deep reinforcement learning for intelligent traffic light control," *IEEE Internet of Things Journal*, vol. 11, no. 17, pp. 28496–28507, 2024.
- [11] D. Sun, A. Jamshidnejad, and B. De Schutter, "A novel framework combining mpc and deep reinforcement learning with application to freeway traffic control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 7, pp. 6756–6769, 2024.
- [12] Z. Ying, S. Cao, X. Liu, Z. Ma, J. Ma, and R. H. Deng, "Privacysignal: Privacy-preserving traffic signal control for intelligent transportation system," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 9, pp. 16290–16303, 2022.
- [13] S. Fang, L. Yang, W.-l. Shang, X. Zhao, F. Li, and W. Ochieng, "Co-operative control model using reinforcement learning for connected and automated vehicles and traffic signal light at signalized intersections," *IEEE Internet of Things Journal*, vol. 99, pp. 1–1, 2025.
- [14] T. Du, B. Wang, and L. Hu, "Single intersection traffic light control by multi-agent reinforcement learning," *Journal of Physics: Conference Series*, vol. 2449, p. 012031, mar 2023.
- [15] Y. Liu, Y. Wang, and Y. Shen, "Demand-aware distributed link allocation in a multilayer heterogeneous satellite network: A game theory approach," *IEEE Internet of Things Journal*, vol. 11, no. 10, pp. 17629–17641, 2024.
- [16] R. Zhu, L. Li, S. Wu, P. Lv, Y. Li, and M. Xu, "Multi-agent broad reinforcement learning for intelligent traffic light control," *Information Sciences*, vol. 619, pp. 509–525, 2023.
- [17] K. Cymerys and M. Oszust, "Attraction–repulsion optimization algorithm for global optimization problems," *Swarm and Evolutionary Computation*, vol. 84, p. 101459, 2024.
- [18] X. Wang, L. Ke, Z. Qiao, and X. Chai, "Large-scale traffic signal control using a novel multiagent reinforcement learning," *IEEE Transactions on Cybernetics*, vol. 51, no. 1, pp. 174–187, 2021.
- [19] T. Chu, J. Wang, L. Codecà, and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE transactions on intelligent transportation systems*, vol. 21, no. 3, pp. 1086–1095, 2019.
- [20] H. Wang, Y. Yu, and Y. Jiang, "Review of the progress of communication-based multi-agent re-inforcement learning," *Scientia Sinica(Informationis)*, vol. 52, no. 5, pp. 742–764, 2022.
- [21] X. Tan, L. Zhou, H. Wang, Y. Sun, H. Zhao, B.-C. Seet, J. Wei, and V. C. M. Leung, "Cooperative multi-agent reinforcement-learning-based distributed dynamic spectrum access in cognitive radio networks," *IEEE Internet of Things Journal*, vol. 9, no. 19, pp. 19477–19488, 2022.
- [22] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, et al., "Soft actor-critic algorithms and applications," *arXiv preprint arXiv:1812.05905*, 2018.
- [23] X. Wu, X. Li, J. Li, P. C. Ching, V. C. M. Leung, and H. V. Poor, "Caching transient content for iot sensing: Multi-agent soft actor-critic," *IEEE transactions on communications. IEEE Communications Society*, vol. 69, no. 9, pp. 5886–5901, 2021.
- [24] D. Li and O. Okhrin, "Modified ddpg car-following model with a real-world human driving experience with carla simulator," *Transportation Research Part C: Emerging Technologies*, vol. 147, p. 103987, 2023.
- [25] L. F. P. D. Oliveira, L. T. Manera, and P. D. G. da Luz, "Development of a smart traffic light control system with real-time monitoring," *IEEE internet of things journal*, vol. 8, no. 5, pp. 3384–3393, 2021.
- [26] L. Zhang, Z. Zhou, B. Yi, J. Wang, C.-M. Chen, and C. Shi, "Edge-cloud framework for vehicle-road cooperative traffic signal control in augmented internet of things," *IEEE Internet of Things Journal*, vol. 12, no. 5, pp. 5488–5499, 2025.
- [27] C. Zhu, M. Dastani, and S. Wang, "A survey of multi-agent deep reinforcement learning with communication," *Autonomous Agents and Multi-Agent Systems*, vol. 38, no. 1, p. 4, 2024.
- [28] C. Li, A. He, G. Liu, Y. Wen, A. T. Chronopoulos, and A. Giannakos, "Rfl-apia: A comprehensive framework for mitigating poisoning attacks and promoting model aggregation in iiot federated learning," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 11, pp. 12935–12944, 2024.
- [29] X. Liang, X. Du, G. Wang, and Z. Han, "A deep reinforcement learning network for traffic light cycle control," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 2, pp. 1243–1253, 2019.
- [30] W. Genders and S. Razavi, "Evaluating reinforcement learning state representations for adaptive traffic signal control," *Procedia Computer Science*, vol. 130, pp. 26–33, 2018.
- [31] N. Xu, G. Zheng, K. Xu, Y. Zhu, and Z. Li, "Targeted knowledge transfer for learning traffic signal plans," *Advances in Knowledge Discovery and Data Mining: 23rd Pacific-Asia Conference, PAKDD 2019, Macau, China, April 14-17, 2019, Proceedings, Part II 23*, pp. 175–187, 2019.
- [32] J. Gao, Y. Shen, J. Liu, M. Ito, and N. Shiratori, "Adaptive traffic signal control: Deep reinforcement learning algorithm with experience replay and target network," *arXiv preprint arXiv:1705.02755*, 2017.
- [33] J. Yang, J. Zhang, and H. Wang, "Urban traffic control in software defined internet of things via a multi-agent deep reinforcement learning approach," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 6, pp. 3742–3754, 2020.
- [34] P. Zhou, X. Chen, Z. Liu, T. Braud, P. Hui, and J. Kangasharju, "Drle: Decentralized reinforcement learning at the edge for traffic light control in the iov," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 4, pp. 2262–2273, 2020.
- [35] H. Ge, D. Gao, L. Sun, Y. Hou, C. Yu, Y. Wang, and G. Tan, "Multi-agent transfer reinforcement learning with multi-view encoder for adaptive traffic signal control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 12572–12587, 2021.
- [36] R. Zhu, S. Wu, L. Li, P. Lv, and M. Xu, "Context-aware multiagent broad reinforcement learning for mixed pedestrian-vehicle adaptive traffic light control," *IEEE Internet of Things Journal*, vol. 9, no. 20, pp. 19694–19705, 2022.
- [37] S. Yang and B. Yang, "A semi-decentralized feudal multi-agent learned-goal algorithm for multi-intersection traffic signal control," *Knowledge-based systems*, vol. 213, p. 106708, 2021.
- [38] L. Yan, L. Zhu, K. Song, Z. Yuan, Y. Yan, Y. Tang, and C. Peng, "Graph cooperation deep reinforcement learning for ecological urban traffic signal control," *Applied Intelligence*, vol. 53, no. 6, pp. 6248–6265, 2022.
- [39] F. Rasheed, K.-L. A. Yau, R. M. Noor, C. Wu, and Y.-C. Low, "Deep reinforcement learning for traffic signal control: A review," *IEEE Access*, vol. 8, pp. 208016–208044, 2020.
- [40] S. Gronauer and K. Diepold, "Multi-agent deep reinforcement learning: a survey," *Artificial Intelligence Review*, vol. 55, no. 2, pp. 895–943, 2021.
- [41] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *International conference on machine learning*, pp. 1861–1870, 2018.
- [42] C. Banerjee, Z. Chen, and N. Noman, "Improved soft actor-critic: Mixing prioritized off-policy samples with on-policy experiences," *IEEE*

- Transactions on Neural Networks and Learning Systems*, vol. 35, no. 3, 2024.
- [43] S. Xiao, Z. Huang, G. Zhang, S. Yang, H. Jiang, and T. Li, "Deep reinforcement learning algorithm of multi-agent based on sac," *Acta Electronica Sinica*, no. 9, pp. 1675–1681, 2021.
 - [44] Z. Woywood, J. I. Wiltfang, J. Luy, T. Enders, and M. Schiffer, "Multi-agent soft actor-critic with global loss for autonomous mobility-on-demand fleet control," *arXiv preprint arXiv:2404.06975*, 2024.
 - [45] Y. Xu, D. Li, and Y. Xi, "Perimeter traffic flow control for a multi-region large-scale traffic network with markov decision process," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 6, pp. 4809–4821, 2024.
 - [46] C. Chi, Y. Wang, X. Tong, M. Siddula, and Z. Cai, "Game theory in internet of things: A survey," *IEEE Internet of Things Journal*, vol. 9, no. 14, pp. 12125–12146, 2022.
 - [47] Y. Jiang, Q. Liu, and Z. Hu, "Actor-critic algorithm with maximum-entropy correction," *Chinese Journal of Computers*, vol. 43, no. 10, pp. 1897–1908, 2020.
 - [48] M. Aslani, M. S. Mesgari, and M. Wiering, "Adaptive traffic signal control with actor-critic methods in a real-world traffic network with different traffic disruption events," *Transportation Research Part C: Emerging Technologies*, vol. 85, pp. 732–752, 2017.
 - [49] F. Xu, F. Xu, J. Xie, C.-M. Pun, H. Lu, and H. Gao, "Action recognition framework in traffic scene for autonomous driving system," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 11, pp. 22301–22311, 2021.
 - [50] H. Wei, C. Chen, G. Zheng, K. Wu, V. Gayah, K. Xu, and Z. Li, "Presslight: Learning max pressure control to coordinate traffic signals in arterial network," *ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1290–1298, 2019.
 - [51] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *arXiv preprint arXiv:1511.05952*, 2015.
 - [52] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wiessner, "Microscopic traffic simulation using sumo," pp. 2575–2582, 2018.
 - [53] P. Singh, B. Hazarika, K. Singh, W.-J. Huang, and T. Q. Duong, "Genai-enhanced federated multiagent drl for digital-twin-assisted iov networks," *IEEE Internet of Things Journal*, vol. 12, no. 5, pp. 4834–4851, 2025.
 - [54] L. Prashanth and S. Bhatnagar, "Reinforcement learning with function approximation for traffic signal control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 12, no. 2, pp. 412–421, 2011.
 - [55] C. Zhou, B. Huang, H. Hassan, and P. Fränti, "Attention-based advantage actor-critic algorithm with prioritized experience replay for complex 2-d robotic motion planning," *Journal of Intelligent Manufacturing*, vol. 34, no. 1, pp. 151–180, 2022.